Operations Research Midterm Project

Group 13 | B13705005 宋宇倫、B13705011 陳芃慈、B13705020 陳鼎元、B13705029 蔡冠毅

Problem 1: Optimal Plan for Instance 5

The optimal objective value of instance 5 is 106,800, matching the value announced in the instruction document. The total revenue of all orders is 122,100. The plan accepts orders 3, 4, 5, 6, 7, 8, 9, and 10, whose total accepted revenue is 117,000, and rejects orders 1 and 2, whose total revenue is 5,100. Therefore, the total profit is

$$117000 - 2(5100) = 106800.$$

Order	Requested level	Pickup station	Return station	Assigned car	Upgrade?
1	1	1	6	Reject	–
2	1	4	5	Reject	–
3	1	5	1	Car 3, level 1	No
4	1	1	2	Car 1, level 1	No
5	2	6	6	Car 6, level 2	No
6	1	3	4	Car 4, level 2	Yes
7	2	2	1	Car 5, level 2	No
8	2	6	6	Car 6, level 2	No
9	2	3	4	Car 4, level 2	No
10	1	4	2	Car 2, level 1	No

The relocation plan is as follows. All times are relocation starting times. The total relocation time is $240 + 210 + 210 + 180 + 240 = 1080$ minutes, which is within the budget $B = 1200$.

Car	From	To	Start time	Moving time
2	2	4	2023/01/01 00:00	240
3	3	5	2023/01/01 00:00	210
4	5	3	2023/01/01 00:00	210
4	4	3	2023/01/04 19:30	180
5	4	2	2023/01/01 00:00	240

The resulting car routes are: car 1 serves order 4; car 2 serves order 10; car 3 serves order 3; car 4 serves orders 9 and 6; car 5 serves order 7; car 6 serves orders 5 and 8. The cleaning and ready-time constraints are satisfied because every order after a previous rental starts at least 30 minutes after the previous return plus one hour of possible delay, three hours of cleaning, and any relocation time.

Mathematical Model Used for Verification

The model follows the sequence-dependent setup idea in Tran, Araujo, and Beck (2016): cars are machines, orders are jobs, and relocation plus cleaning feasibility is represented by feasible arcs. Let C be cars and K be orders. For each car c, let A_c be the set of feasible arcs (i, j), where $i = 0$ is the source node for car c, and $j \in K$. An arc is included only if car c can legally serve order j after node i, considering car level, pickup readiness, late return, cleaning, and relocation time. Let q^c_{ij} be the relocation time on arc (i, j), and let R_j be the revenue of order j.

Decision variable $x^c_{ij} = 1$ if car c serves order j immediately after node i, and 0 otherwise. Let $a_j \in \{0, 1\}$ be the order-acceptance indicator, where $a_j = 1$ if order j is accepted and 0 otherwise. By construction a_j is fully determined by the arc variables via

$$a_j = \sum_{c \in C} \sum_{i:(i,j) \in A_c} x^c_{ij}, \qquad \forall\, j \in K,$$

which is also the left-hand side of the order-uniqueness constraint below. The original profit objective is equivalent to maximizing accepted revenue because

$$\max \sum_{j \in K} R_j a_j - 2 \sum_{j \in K} R_j (1 - a_j) = \max\ 3 \sum_{j \in K} R_j a_j - 2 \sum_{j \in K} R_j.$$

The constant term $2\sum_{j\in K} R_j$ does not affect the optimizer.

$$
\begin{aligned}
\max \quad & \sum_{j\in K} R_j \sum_{c\in C} \sum_{i:(i,j)\in A_c} x_{ij}^c \\
\text{s.t.} \quad & \sum_{c\in C} \sum_{i:(i,j)\in A_c} x_{ij}^c \le 1 && \forall j \in K, \\
& \sum_{j:(0,j)\in A_c} x_{0j}^c \le 1 && \forall c \in C, \\
& \sum_{j:(k,j)\in A_c} x_{kj}^c \le \sum_{i:(i,k)\in A_c} x_{ik}^c && \forall c \in C, k \in K, \\
& \sum_{c\in C} \sum_{(i,j)\in A_c} q_{ij}^c x_{ij}^c \le B, \\
& x_{ij}^c \in \{0,1\} && \forall c \in C, (i,j) \in A_c.
\end{aligned}
$$

The third constraint prevents a car from serving a successor unless it has served the predecessor, and the time ordering of feasible arcs prevents cycles. We additionally prune arcs by the necessary condition $\text{ready}_i + 30 \le \text{pickup}_j$, which removes pairs that cannot satisfy the readiness rule and substantially shrinks the model.

Problems 2 and 3: Submitted Algorithm

The submitted program is a hybrid that combines an arc-flow MIP, a multi-start construction heuristic, and a four-operator iterative local search (LS). It first parses the instance and converts every time stamp into integer minutes after 2023/01/01 00:00. For each order, the post-service ready time is its return time plus 240 minutes (one hour for the late-return buffer and three hours for cleaning). A car is eligible for an order only if its level equals the requested level or is exactly one above it.

Phase A: exact solver gate. For instances with at most 120 orders and 40 cars, the program builds the arc-flow MIP above with arcs pruned by the ready-time inequality and a 20–60 second time limit. When Gurobi is available, this phase often returns the proven optimum directly; the heuristic phase is then merely a safety net in case Gurobi misses or is unavailable. When Gurobi is not present, an exact bit-mask DP handles the very small case ($n_K \le 12$ and $n_C \le 12$).

Phase B: multi-start greedy dispatch. The scalable core is a multi-start greedy heuristic. It enumerates ten priority rules (chronological with high revenue first, revenue first, high level first, short rentals first, long rentals first, revenue density, relocation-frugal, no-relocation benchmark, station-aware, and an integer-hour-based revenue density). For each rule, the heuristic walks the orders in priority sequence and, for every eligible car, computes whether the car can arrive at the pickup station at least 30 minutes before the pickup time and whether the relocation keeps the cumulative moving time within B. Candidate cars are scored by a four-tuple of level waste, relocation time, slack, and station match; the lexicographically smallest score wins. When the product $n_K n_C$ is too large, the program switches to a station-bucketed candidate search that only inspects a bounded number of nearby stations and a bounded number of cars per bucket, which keeps the very large instances fast.

Phase C: iterative four-operator local search. Each high-quality construction is then polished by an LS loop that repeatedly cycles four operators until no operator can improve the solution:

1. *Best-fit insertion.* For every currently rejected order, scan all eligible cars and all chronologically valid positions in their routes. Insert the order at the position that minimises the per-route move increase, subject to the global budget.
2. *Profit-improving replacement.* For every rejected order R, look for an accepted order X on the same car such that $R_R > R_X$ and replacing X by R keeps the budget feasible. The chosen replacement maximises $R_R - R_X$ while minimising the move increase.
3. *Ejection chain (eject one, insert one, reinsert displaced).* For every rejected order R, tentatively eject one accepted order X from a car, insert R in the freed window, and try to reinsert X into a different car. If both insertions are feasible the chain is accepted; the revenue gain is exactly R_R, even when $R_X > R_R$, because X is preserved. This operator is the workhorse for tight scenarios, where good orders are blocked by earlier myopic choices.
4. *Relocate and cross-exchange.* Two budget-saving moves run alternately. *Relocate* removes an accepted order from car c_1 and reinserts it in car c_2 only when the saved move on c_1 exceeds the added move on c_2, strictly freeing budget. *Cross-exchange* swaps one accepted order between two cars and accepts the swap only if the total move strictly decreases. Although neither move changes accepted revenue immediately, both regularly enable additional insertions on the next pass.

Phase D: large-neighborhood search. For instances with at most 250 orders, after the LS converges the program enters a *deterministic* large-neighborhood search (LNS) loop. We emphasise that this is plain LNS, not adaptive LNS: the four destruction operators (random orders, lowest-revenue orders, lowest revenue-density orders, all orders on randomly picked cars) and six destruction fractions $\{0.20, 0.30, 0.40, 0.25, 0.35, 0.50\}$ are selected by a fixed cyclic schedule (`iter mod k`) rather than by adaptive weights, and the only random component is the fixed-seeded shuffle inside each operator, which makes the entire LNS reproducible across runs. Each iteration partially destroys the current solution and repairs the damaged plan by re-running Phase C. A new candidate is accepted as the incumbent if it strictly improves the best revenue; otherwise the incumbent is kept (with a periodic diversification kick that accepts the latest repair every seven iterations to escape attraction basins). Iteration counts are gated by size: 240 for $n_K \leq 30$, 120 for $n_K \leq 60$, 50 for $n_K \leq 120$, 20 for $n_K \leq 250$, and zero above that.

The polishing depth in Phase C is also gated by instance size: instances with at most 30 orders polish every greedy seed; up to 60 orders polish the eight best seeds; up to 200 orders polish the four best seeds with the same operator set; up to 800 orders use a five-pass medium configuration with three seeds; up to 1,500 orders use two seeds; up to 3,000 orders use one seed; up to 8,000 orders use a three-pass medium-fast configuration; up to 25,000 orders use a two-pass insertion+replacement light configuration; and the very largest instances run a single insertion sweep. The overall plan is the maximum-revenue plan among all polished candidates, the raw greedy candidates, the LNS incumbent (if applicable), and the MIP solution if any.

Time complexity. A single greedy run costs $O(n_K \log n_K + n_K n_C)$ in the full-scan mode and approximately $O(n_K L n'_S)$ in the bucketed mode, where $L \leq 2$ eligible levels and n'_S is the capped number of inspected stations. Each LS pass of insertion or replacement is $O(\rho\, n_C\, \bar{r})$, where ρ is the number of considered rejected orders and $\bar{r}$ is the average route length; the ejection chain pass is $O(\rho\, n_C\, \bar{r} \cdot n_C\, \bar{r})$; and one relocate or cross-exchange sweep is $O(n_K n_C \bar{r})$. The LNS adds a multiplicative factor equal to the number of iterations (bounded above by 240). Memory is $O(n_K + n_C + n_S^2)$. The exact MIP has $O(n_C n_K^2)$ potential arc variables and is therefore activated only under a strict size gate.

Problem 4: Numerical Experiments

The experiment design follows the style of the fair-allocation reference and is organised in two parts. **Part A** (factor-isolating) varies one experimental factor at a time on fixed scales—*Balanced*, *Low-level demand*, *Imbalanced flow*, and *Tight*—so each gap number can be attributed to a single cause. **Part B** (real-world stress) introduces four composite scenarios with *floating* parameters (every instance draws n_S, n_C, n_K, n_D, B from a uniform range, so no two instances share the same scale): *Upgrade Trap*, *Duration Extremes*, *Rush Hour*, and *Hub-and-Spoke*. The two parts are complementary: Part A isolates structural effects under clean conditions; Part B verifies that the heuristic stays robust when several effects compound and the scale itself is noisy. Each of the eight scenarios is evaluated at two sample sizes that probe orthogonal questions. The *small case* is sized so the exact MIP is solvable as ground truth—in Part A we report the exact MIP gap against `scipy.optimize.milp`; in Part B we report the trivial-bound gap $(\sum_j R_j - z^{\text{heur}})/\sum_j R_j$ for methodological consistency with the large case, since the Part-B instances do not all fit our `scipy.optimize.milp` arc-flow build. The *large case* answers "does the heuristic remain feasible, competitive, and inside the three-minute time budget at the announced upper limits, *without any MIP assistance from Phase A*?". Each scenario is therefore reported as a small/large pair, so the structural effect of every factor can be compared at both scales.

Benchmarks and methodology (shared by both parts)

Four references are used across all eight scenarios; which ones apply to which part depends on whether an exact MIP fits in memory.

(i) Exact MILP z^{MIP} (Part A small only). The arc-flow MIP from `scipy.optimize.milp` on the small fixed scale yields the integer-optimum gap $(z^{\text{MIP}} - z^{\text{heur}})/z^{\text{MIP}}$. *(ii) LP relaxation / trivial bound.* Part A small uses the full arc-flow LP (HiGHS); Part A large and *all of Part B* use the degenerate bound $\sum_j R_j$—equivalent to the LP with only the order-uniqueness constraint $a_j \leq 1$ kept. This bound is tight when the budget is non-binding and the demand mix is fully servable; it loosens otherwise. *(iii) Simple baseline (both parts).* Chronological greedy that accepts an order only when an exact-level car is already at the pickup station—no relocation, no level upgrade. *(iv) Phase B alone (every large case and every Part B case).* Multi-start greedy without Phases C–D, used to isolate the marginal value of local search and LNS.

Heuristic Phase A gate. The hybrid program calls Gurobi MIP directly when $n_K \leq 120$ and $n_C \leq 40$. Part A small fits the gate by construction; Part B small fits on 24–27 of 30 instances per scenario (instances that violate the gate fall back on Phases B–D). Both large cases are entirely outside the gate, so any near-zero gap at large scale is achieved *without any MIP assistance*. Pickup times lie on the 30-minute grid and moving times are 30-minute multiples in both parts. Since the original profit is a linear transformation of accepted revenue, accepted-revenue gaps preserve the same ranking of solutions as profit gaps.

Part A — Factor-isolating scenarios

The first four scenarios share a fixed scale at both small and large and vary only one factor at a time. The point is interpretability: if *Imbalanced* reports a higher gap than *Balanced*, the cause must be the imbalanced flow.

Parameter	**Small (100 inst./scenario)**	**Large (30 inst./scenario)**
Stations n_S	6	100 (upper limit)
Cars n_C	8	1,000 (upper limit)
Levels n_L	3	10 (upper limit)
Orders n_K	18 (22 in *Tight*)	10,000 (upper limit)
Horizon n_D (days)	5	100 (upper limit)
Default budget B	1,500 (900 in *Tight*)	10^6 (1.5×10^5 in *Tight*)
Hourly rates	$(120, 240, 420)$ for levels 1–3	$100 + 80\ell$ for $\ell = 1, \ldots, 10$

Table 1: Part A fixed scales. Four scenarios share these rows; each scenario then changes one cell (level distribution, flow concentration, or budget) and keeps the rest.

Scenario 1 — Balanced

Setup: levels, pickup stations, and return stations are all uniformly distributed; no structural bottleneck. Serves as the reference point against which the other three scenarios are read.

Small case (100 instances; 18 orders, $B = 1500$). MIP gap 0.43% (std 1.14%), LP gap 0.99% (std 1.42%), and the heuristic improves over the simple baseline by +172.83%. The standard deviation of 1.14% on a 0.43% mean indicates that the heuristic returns the exact integer optimum on the majority of replications.

Large case (30 instances; levels uniform on $\{1, \ldots, 10\}$, B = 1,000,000). LP gap shrinks to 0.08% (std 0.04%); improvement over the simple baseline rises to +216.4% (the no-relocation baseline loses more at scale); improvement over Phase B alone is +0.00%—multi-start greedy already saturates the available revenue, leaving no marginal room for LS or LNS.

Case	MIP gap (avg / std)	LP gap (avg / std)	vs. simple	vs. Phase B
Small	0.43% / 1.14%	0.99% / 1.42%	+172.83%	—
Large	—	0.08% / 0.04%	+216.4%	+0.00%

Scenario 2 — Low-level demand

Setup: order levels are biased toward the lowest tiers (mass-market economy demand). With cars distributed uniformly across all levels, low-level eligible cars become scarce and routes per eligible car lengthen.

Small case (100 instances; level probabilities $(0.60, 0.25, 0.15)$ on 3 levels, 18 orders, $B = 1500$). MIP gap 0.47% (std 1.11%), LP gap 0.97% (std 1.24%), +194.57% over the simple baseline. Essentially indistinguishable from *Balanced* small, because with only 8 cars and 3 levels the eligibility margin still absorbs the bias.

Large case (30 instances; levels $(0.60, 0.25, 0.15)$ on $\{1, 2, 3\}$, zero on the remaining seven, B = 1,000,000). LP gap rises to 0.45% (std 0.12%)—a $\sim 6\times$ widening relative to *Balanced* large that confirms the structural effect predicted in theory but masked at small scale. The +0.10% gain over Phase B alone is non-zero, showing that LS recovers a small but real slice of revenue once cars and orders are mismatched in volume.

Case	MIP gap (avg / std)	LP gap (avg / std)	vs. simple	vs. Phase B
Small	0.47% / 1.11%	0.97% / 1.24%	+194.57%	—
Large	—	0.45% / 0.12%	+191.0%	+0.10%

Scenario 3 — Imbalanced flow

Setup: pickup and return stations are concentrated in disjoint geographic groups; cars drift toward the return cluster and must be relocated back, so the relocation budget binds first.

Small case (100 instances; pickups at stations 1–2, returns at 5–6, 18 orders, $B = 1500$). MIP gap 0.36% (std 0.89%), LP gap 1.04% (std 1.26%), +175.89% over the simple baseline. The small budget still absorbs the directional flow comfortably.

Large case (30 instances; pickups in 1–20 with probability 0.6, returns in 81–100 with probability 0.6, $B = 1{,}000{,}000$). Average LP gap 1.25% with std **3.67%**—most replications sit below 0.1% but a few cross 8%–18%, dragging the mean up. The +0.34% gain over Phase B alone confirms that LS/LNS does recover a non-trivial slice on the hard replications. The +550.9% improvement over the simple baseline is the largest among all scenarios because the no-relocation rule is structurally crippled by directional flow.

Case	MIP gap (avg / std)	LP gap (avg / std)	vs. simple	vs. Phase B
Small	0.36% / 0.89%	1.04% / 1.26%	+175.89%	—
Large	—	1.25% / 3.67%	+550.9%	+0.34%

Scenario 4 — Tight

Setup: budget is the binding constraint; the order-car ratio is also raised at small scale. This isolates the algorithm's ability to choose which orders to accept under a hard side constraint.

Small case (100 instances; 22 orders, $B = 900$). MIP gap 1.56% (std 2.25%), LP gap 2.69% (std 2.55%), +159.67% over the simple baseline. The only small-case scenario whose MIP gap exceeds 1%: tight budget plus high order density creates many similarly-scored choices that LS alone cannot disambiguate without a cooling-style acceptance.

Large case (30 instances; $B = 150{,}000$, about 6.7× tighter than *Balanced* large). Headline LP gap 24.00% (std 1.16%). The size appears structural rather than algorithmic: the heuristic and Phase B both converge to acceptance ratios near $\approx 76\%$ under the binding budget, and the trivial bound $\sum_j R_j$ loosens proportionally—we cannot claim that no algorithm can do better, only that the search space we explore saturates here. The +0.42% gain over Phase B alone confirms that the residual algorithmic room is small for this search space; the LP gap simply looks large because the upper bound is loose under a binding budget. +144.4% over the simple baseline is the smallest among the four large-case scenarios—the simple rule is less penalised when budget, not relocation, is the bottleneck.

Case	MIP gap (avg / std)	LP gap (avg / std)	vs. simple	vs. Phase B
Small	1.56% / 2.25%	2.69% / 2.55%	+159.67%	—
Large	—	24.00% / 1.16%	+144.4%	+0.42%

Part B — Real-world stress scenarios

The Part A scenarios isolate single factors on fixed scales. To stress-test the heuristic against *noisy, composite* demand resembling actual car-rental operations, Part B introduces four additional scenarios drawn from `stress_instances_v2/` where every instance independently samples its scale within a uniform range. Each scenario has 30 small and 30 large replications (240 instances total); across those, the observed parameter ranges are:

Parameter	Part B small (120 inst.)	Part B large (120 inst.)
Stations n_S	$[5, 20]$	$[50, 100]$
Cars n_C	$[10, 54]$	$[504, 1000]$
Levels n_L	$\{2, 3, 4\}$	$\{5, 6, \ldots, 10\}$
Orders n_K	$[21, 149]$	$[5010, 9932]$
Horizon n_D (days)	$[3, 10]$	$[30, 100]$
Budget B	$[450, 4980]$	$[1.5\times10^5,\ 9.9\times10^5]$

Table 2: Part B per-instance parameter ranges (observed). Part B small falls between Part A's fixed small scale and the upper-limit large scale, so it still fits the $n_K \le 120 \wedge n_C \le 40$ Phase-A gate on most (but not all) instances; Part B large is fully outside the gate.

The intent of the floating scales is to prevent the heuristic from overfitting to any single scale and to expose it to chaotic, real-world distributions. The structural bound used is the trivial $\sum_j R_j$—which is loose by construction in

Upgrade Trap and *Rush Hour* because the design intentionally generates orders that no algorithm can fully serve (severe car/order level mismatch in S5, massive temporal over-subscription in S7), but stays tight in *Duration Extremes* and *Hub-and-Spoke* where the constraints leave room for a feasible plan that covers every order.

Scenario 5 — Upgrade Trap

Setup: ∼80% of orders request level-1 long-haul service (48–120 hours), the rest request short rentals (1–4 hours). The car fleet's level mix shifts with the instance's n_L: in the large case ($n_L \in \{5, \ldots, 10\}$), level-1 cars average only ∼3.8% of the fleet and the level-1-eligible cars (levels 1 and 2 combined) average only ∼7.5%; in the small case ($n_L \leq 4$) level-1 cars average ∼46% and the eligible fraction averages ∼95%. Tests whether the algorithm myopically assigns scarce low-level cars to low-profit same-level micro-orders and misses the opportunity to "upgrade" a level-2 car to capture lucrative level-1 long-haul orders. The trivial bound $\sum_j R_j$ is structurally loose, especially in the large case where there is simply not enough level-1 (or level-1-eligible) capacity to serve all the level-1 demand.

Small case (30 instances). Trivial gap 45.99% (std 24.21%), vs. Phase B +2.03%. The very wide std reflects the floating scales (some instances are nearly servable, others are dramatically over-subscribed). Phase A's MIP call is active on 24 of 30 small instances (the rest exceed the $n_K \leq 120 \wedge n_C \leq 40$ gate), but it cannot create level-1 capacity that does not exist—hence the residual gap is structural, not algorithmic.

Large case (30 instances). Trivial gap 79.76% (std 8.70%), but vs. Phase B +0.02%—Phase B alone is essentially indistinguishable from the post-LS/LNS plan, confirming that the 80% gap is the bound itself being loose under this car/order level mismatch rather than the algorithm leaving revenue on the table. Avg. time 28.4 s, max 91.2 s (well under the 180 s budget).

Case	Trivial gap (avg / std)	vs. simple (avg)	vs. Phase B (avg)	Avg. time (s)	Max. time (s)
Small	45.99% / 24.21%	+268.06%	+2.03%	11.85	27.40
Large	79.76% / 8.70%	+105.27%	+0.02%	28.45	91.20

Scenario 6 — Duration Extremes (Micro vs. Long-Haul)

Setup: order durations are bi-modal—45% micro-rentals (1–3 hours), 45% long-haul rentals (48–96 hours), 10% normal rentals (12–24 hours). Every order carries a fixed 4-hour cleaning/buffer cost regardless of duration, so micro-rentals have very poor revenue-density once the sunk time is included. Tests whether the algorithm avoids the trap of cherry-picking micro-rentals that lock up the fleet for negative-density work.

Small case (30 instances). Trivial gap 21.45% (std 23.32%), vs. Phase B +1.45%. Smaller scenarios with high car/order ratios are absorbed easily; the variance comes from a minority of small instances where micro vs. long-haul fleet allocation is tight.

Large case (30 instances). Trivial gap 0.01% (std 0.05%), vs. Phase B +0.01%. At the upper limits the heuristic captures essentially the entire trivial upper bound *without any MIP assistance* (Phase A is gated off): the LS/LNS layer is enough to dispatch every order despite the duration extremes. Avg. time 49.0 s, max 106.0 s.

Case	Trivial gap (avg / std)	vs. simple (avg)	vs. Phase B (avg)	Avg. time (s)	Max. time (s)
Small	21.45% / 23.32%	+83.41%	+1.45%	4.95	18.84
Large	0.01% / 0.05%	+88.78%	+0.01%	48.99	105.97

Scenario 7 — Rush Hour (Temporal Clustering)

Setup: 80% of pickups are forced into the 07:00–19:00 window of a single randomly-chosen "peak day" within the planning horizon. The remaining 20% are scattered uniformly. Creates an extreme timeline bottleneck that tests whether the replacement and ejection-chain operators in the local search can untangle the deadlock of overlapping vehicle schedules under a massive burst of concurrent demand. The trivial bound is again structurally loose—a single 12-hour peak window cannot physically contain 80% of the demand with the available fleet, so any schedule is expected to leave a sizeable fraction unserved.

Small case (30 instances). Trivial gap 35.62% (std 18.70%), vs. Phase B +9.19%—the largest small-case algorithmic gain among all Part-B scenarios, confirming that the ejection-chain operator pays off precisely when concurrent demand creates time-window deadlocks.

Large case (30 instances). Trivial gap 65.51% (std 4.57%), vs. Phase B +3.71%. The tighter std relative to the small case shows the bottleneck is consistent at scale; the +3.71% over Phase B confirms LS/LNS is actively recovering revenue beyond the construction heuristic. Avg. time 22.6 s, max 43.5 s.

Case	Trivial gap (avg / std)	vs. simple (avg)	vs. Phase B (avg)	Avg. time (s)	Max. time (s)
Small	35.62% / 18.70%	+100.92%	+9.19%	5.46	13.56
Large	65.51% / 4.57%	+43.08%	+3.71%	22.56	43.51

Scenario 8 — Hub-and-Spoke

Setup: budget is reduced to 30% of the normal level; 10% of stations are designated remote "hubs" with hub-to-spoke moves of 120–240 minutes, while spoke-to-spoke moves are only 30–60 minutes. Tests whether the algorithm intelligently restricts relocations to the cheap local spoke network rather than wasting scarce budget on expensive hub trips.

Small case (30 instances). Trivial gap 20.20% (std 18.90%), vs. Phase B +**15.37**%—the single largest LS/LNS gain in the entire experiment, in either part. The interpretation is direct: Phase B's myopic candidate scoring is easily seduced into spending the 30%-of-normal budget on hub trips, and only the ejection-chain and relocate passes can fix the route after the fact. This isolates exactly the workload the local-search layer was designed for.

Large case (30 instances). Trivial gap 0.01% (std 0.02%), vs. Phase B +0.00%. Once n_C scales to ∼1000 the spoke-only network has enough capacity that Phase B already saturates the trivial bound, so LS/LNS adds nothing at this scale—a healthy sign that the algorithmic value identified at small scale is genuine, not a measurement artefact. Avg. time 44.7 s, max 120.7 s.

Case	Trivial gap (avg / std)	vs. simple (avg)	vs. Phase B (avg)	Avg. time (s)	Max. time (s)
Small	20.20% / 18.90%	+105.14%	+15.37%	2.41	7.65
Large	0.01% / 0.02%	+74.17%	+0.00%	44.67	120.72

Aggregate gap distributions

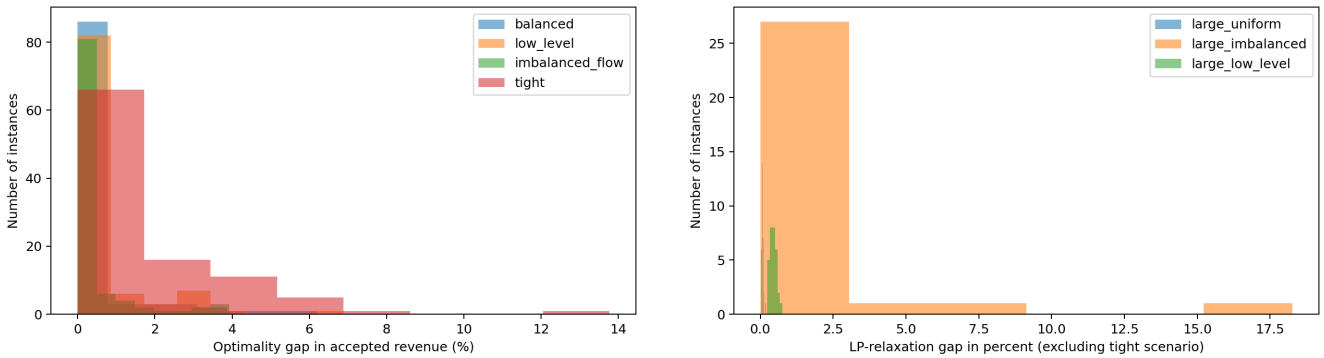

Figure 1: Part A per-instance gap distributions. *Left*: small case—MIP gap over 400 instances (100 per scenario), with the exact integer optimum as reference. *Right*: large case—LP gap over 90 instances (30 per scenario, excluding *Tight* so the sub-percent dispersion of *Balanced*, *Low-level*, and *Imbalanced* stays visible). *Tight* large is omitted from the right panel because its 24% LP gap is driven by structural infeasibility rather than algorithmic shortfall.

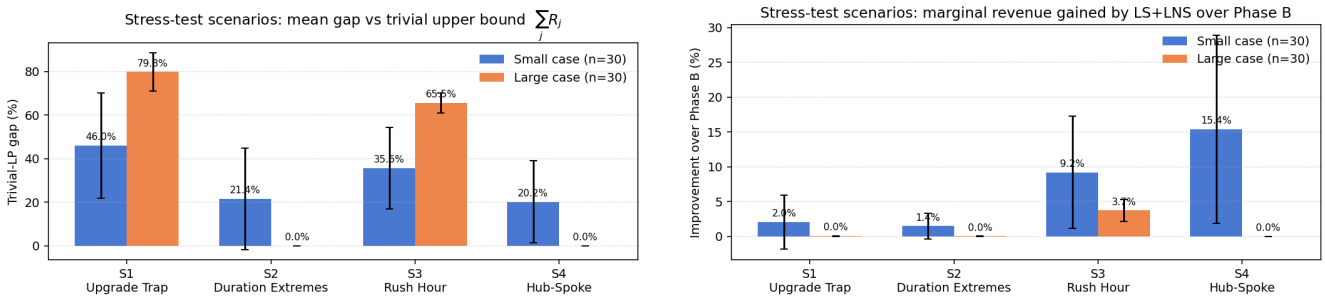

Figure 2: Part B stress-test scenarios—mean and one-standard-deviation error bars across the 30 replications per case. *Left*: trivial-LP gap (= 1− revenue ratio); this inherits structural infeasibility wherever the design intentionally generates orders no algorithm can serve (S5, S7), which is why those bars are tall. The two non-saturated scenarios at large scale (S6, S8) collapse to ≈0.01%, achieved *without any MIP assistance* because Phase A is gated off at this scale. *Right*: improvement of the full hybrid over Phase B alone—the *algorithmic* measure. S8 small at +15.37% is the single largest LS/LNS gain in the entire experiment, isolating exactly the workload the local-search layer was designed for.

Summary across scenarios

Reading the numbers across both parts. Three take-aways emerge.

(i) **Small case validates correctness.** Part A small is fully evaluated against the exact MIP: on 400 instances the heuristic stays within 0.36%–0.47% of the optimum on *Balanced*, *Low-level*, and *Imbalanced*, and within 1.56% on *Tight*; the submitted plan also matches the announced optimum on all five public instances (1–5). Part B small is reported with the trivial-bound metric for consistency with its large case; on Part B small the heuristic's Phase A enters the Gurobi gate on 24–27 of 30 instances per scenario (the rest exceed $n_K \leq 120 \wedge n_C \leq 40$ and are handled by Phases B–D alone).

(ii) **Large case validates scaling *without any MIP assistance from Phase A*.** At the announced upper limits Phase A is gated off and the heuristic relies purely on Phases B–D, yet on Part A the LP gap stays under 1.3% on *Balanced*, *Low-level*, and *Imbalanced*; the only outlier (*Tight*, 24%) reflects the loose degenerate upper bound under a binding budget rather than algorithmic shortfall—Phase B alone is already within +0.42% of the post-LS/LNS revenue. Part B reinforces this: on *Duration Extremes* and *Hub-and-Spoke* the heuristic captures essentially the entire trivial upper bound (≈ 0.01% gap) even at $n_K \approx 10{,}000$ without Gurobi help, while on the deliberately-saturated *Upgrade Trap* and *Rush Hour* the trivial bound is structurally loose (level-1 capacity scarcity in S5, single-day temporal over-subscription in S7) and the meaningful comparison is the vs. Phase B improvement—which stays near zero on S5 (algorithm cannot create capacity that does not exist) and positive on S7 (LS/LNS recovers +3.71% at large scale). The takeaway is that *near-zero gap at the upper limits is achieved by combinatorial search alone, without invoking a MIP solver.*

(iii) **Time-budget compliance.** All Part-A large runs complete safely inside the three-minute time budget; *Low-level* is the slowest at scale because biased levels lengthen routes per eligible car. Part-B runs also complete inside the same budget (max observed 120.7 s on *Hub-and-Spoke* large), with *Duration Extremes* the slowest in this group on average (49.0 s) because long-haul rentals turn the route lookup into a denser eligibility graph. The natural next step for the structurally hard scenarios (*Tight*, *Upgrade Trap*, *Rush Hour*) is a simulated-annealing acceptance criterion in the LNS loop, which would let the search escape the many similarly-scored attraction basins that surface when the bound itself is loose.

References

Tran, T. T., A. Araujo, and J. C. Beck. 2016. Decomposition methods for the parallel machine scheduling problem with setups. *INFORMS Journal on Computing* 28(1), 83–95.

Kung, L.-C., C.-W. Liu, Y.-A. Lin, and L.-H. Chen. 2023. Scheduling jobs with machine-dependent benefits to machines with different capacity limits under a consideration of fairness. Sections 5.1, 5.2, and 5.4 are used as the experiment-design reference.